

An Evaluation of The Performance of Lightweight Language Models on Zero-Shot NL2SQL tasks

Tom Fan
Carleton University
Ottawa, Canada
tomfan3@cmail.carleton.ca

ACM Reference Format:

Tom Fan. 2025. An Evaluation of The Performance of Lightweight Language Models on Zero-Shot NL2SQL tasks. In . ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 ABSTRACT

Since their introduction as a consumer product in 2022, transformer-based large language models have been used for automating many kinds of digital tasks. This includes natural language to code and SQL tasks, which have exploded in research and development in the last few years. While a large proportion of LLM use is likely done using proprietary, closed source, and large footprint foundation models hosted by SAAS providers like Anthropic or OpenAI, an increasing amount of open-source models are also entering the market. These models have demonstrated promise on several different kinds of tasks, such as code generation using Python [9]. However, there is limited research on how they perform in other common coding-adjacent tasks, including NL2SQL. While there is an emerging wealth of assessment on how larger models like OpenAI's closed source GPT series perform on NL2SQL tasks in various configurations[13][11], the performance of these small models' smaller cousins in non-pretrained configurations is still an under-explored area. In this paper, we aim to provide an overview of how several such opensource models perform when faced with NL2SQL tasks on the Spider-Dev question dataset.

2 INTRODUCTION

In recent years, there has been a significant amount of high performing open source LLMs available from both research institutions and tech companies [14]. While many of the top-ranking models require expensive and complex hardware setups to run as their parameter counts run in the hundreds of billions or even trillions, many are either small footprint by design or distilled versions of larger source models.

In these cases, consumer level hardware should be able to run the models using tools like localLlama, enabling their use offline and without dependency on external SAAS providers (for purposes of this paper, we define consumer level hardware to be current

generation Nvidia RTX chips or other equivalents, able to comfortably run models in the 20B parameter range or smaller at 8 or 16b quantization).

These LLMs have shown promise in their ability to solve an array of tasks, including completing test sets for code generation such as HumanEval [9][6]. In these experiments, the authors were able to push these models to perform disproportionately well to how large their footprints are, indicating that these smaller LLMs are able to consistently solve simple, well defined tasks and generate effective code.

At the same time, there has been significant development in the field of NL2SQL tools. While previous generations of NL2SQL tools depended on hardcoded rules or deep neural networks [11], the emergence of LLMs has enabled a whole new paradigm of NL2SQL tools based on them. In fact, in their MVP state, a LLM itself can serve as a NL2SQL tool on its own.

While there are many evaluations of the potential of LLMs for NL2SQL tasks, many of these depend on the aforementioned large, closed-source, proprietary models [18][11][13]. At the same time, evaluations of the performance of lightweight, open source models are few in number. Therefore, in this paper we conduct an evaluation asking one guiding question - how well do lightweight LLMs such as Llama 8b or Qwen3 perform when presented with NL2SQL tasks in a zeroshot format?

To answer this guiding question, we took a sample of 8 such models in the 0-20b parameter range and evaluated their performance on the SPIDER-dev NL2SQL question set. Using a zero-shot method with a short context length, we built a testbench[19] to evaluate these opensource models using Novita's online AI service and hardware rental service [2]. This allowed us to run each model on the cloud with consistent hardware, while still being able to evaluate token efficiency and getting results equivalent to what we would see running these models on a local system.

Then, we gave each model we evaluated all 1034 of the SPIDER-dev NL2SQL questions, and recorded their responses. Models were not given pre-training or any other context. We found that while all of these models can zero-shot these NL2SQL problems roughly $\frac{1}{2}$ to $\frac{2}{3}$ of the time in terms of execution accuracy, current small opensource LLMs are still not powerful enough to allow this type of use case consistently.

3 TESTING METHODOLOGY AND EXPERIMENTAL DESIGN

We started our experiment with the objective of evaluating the performance of lightweight open source models on NL2SQL tasks. In order to do so, we used SPIDER-dev as our test set (taoyds/spider) [20], which is a commonly used benchmark for evaluation of NL2SQL

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

tools. Then, we evaluated the extracted SQL from the LLMs' responses using the accompanying taoyds/test-suite-sql-eval tool [22], as per the recommendations of SPIDER's authors [20]. To support these requirements, we built a testbench[19] that would serve as a bridge and data recording tool for SPIDER, the evaluator, as well as Novita, our LLM provider.

While the general architecture of our testing system is fairly straightforward, tradeoffs had to be made in experiential design in several areas. These primarily happened in terms of model selection, test set selection, and prompting strategy and arose from a need for testing consistency and efficiency.

Model selection

Our experimental data incorporates testing from 8 different open source LLMs, each with a parameter count $\leq 30b$. The basic criteria for evaluation included the following requirements:

- Models must be open source, or otherwise downloadable and usable for a user locally at no charge and with no external services. This constrains the selected models to effectively what can be found on HuggingFace.
- Models cannot be fine-tuned specifically for NL2SQL tasks, as this would result in imbalanced results and go against the core evaluation of small, general purpose models being used for NL2SQL tasks. Other fine tuning, however, would not be a deal breaking factor for selection.
- Models must be able to run locally on consumer level hardware. While that which constitutes the category of consumer level hardware is subjective to the end user, for the purposes of this paper we have defined it as current generation consumer CPUs (Intel i series, AMD Ryzen, etc) alongside current generation Nvidia RTX graphics processors or equivalent. This constrains the size of the model to 20-30GB total, as most local LLMs will require $\geq 2GB$ of memory for KV cache or buffer space.

Using this selection criteria, we selected and tested a list of these 8 models available publically:

- llama 3.2 3b instruct [16]
- Qwen 3 4b [4]
- Qwen 2.5 7b instruct [3]
- llama 3.1 8b [15]
- Qwen 3 8b [5]
- Deepseek R1 distill 14b [1]
- GPT OSS 20b [17]
- Gemma 3 27b [8]

At either 8 or 16b quantization, these models should be able to fit in the 32GB of VRAM provided by an RTX 5090, the current flagship model of Nvidia's consumer GPU lineup. These models come from a variety of different developers, have a wide range of parameter counts, and an assortment of architectures (MOE, distilled, etc).

Although we did tests for Qwen's 30b coder and 32b models, these were excluded from testing as a VM instance running what we defined as "consumer level hardware" was not able to produce responses efficiently using these two models as-is.

Additionally, we ran the same tests using Google's Gemma3-12b-it model. We chose to exclude this from testing as it delivered parsable SQL only 68% of the time, and its results effectively broke

the NL2SQL response tester. This leaves it with a score of 0 in every category in our tests, and did not deliver any meaningful insights.

Test set selection

For this paper, we selected the SPIDER-dev [20] NL2SQL dataset as the test set for our models. This was selected from a list of candidates including SPIDER-dev, SPIDER, SPIDER2, and BIRD. Even though these are all well designed, commonly used NL2SQL question sets, we opted to use solely SPIDER-dev for several key reasons.

The full SPIDER dataset contains 10,181 questions [21]. Even though we used a cloud provider to accelerate our inference with better hardware vs running locally, each run of the 1,034 questions in the SPIDER-dev dataset took over 12 hours on average. For a dataset 10x larger, this means that all 9 models would have taken over a month to return a full set of results, from running inference to extracting and running SQL, to recording the results to persistent storage.

The SPIDER2 dataset [10] also contains 10,000+ questions, as well as a significantly higher level of question difficulty. In preliminary testing, the smallest of the models quickly ran out of context window and were therefore unable to deliver a complete SQL query, both breaking the test platform and effectively eliminating this dataset from consideration. Similar issues exist in the BIRD dataset as well [12].

Thus by elimination, the SPIDER-dev question dataset was the last remaining contender under consideration. It contains a variety of databases, question types, and difficulty levels, meaning that even though it is significantly smaller in scope to the other datasets considered, testing a model using it would still lead to meaningful insights into how they performed in different scenarios. Additionally, since an already existing query evaluation tool was written for the SPIDER questions[22], we could leverage it to only need to store our LLM responses and utilize the output from the tool itself.

Prompting Strategy

Every benchmark of LLMs on any task requires a prompting strategy. This must be unified across all models evaluated, since we want all variables besides the model type to be as consistent as possible. Since our goal was to evaluate zero-shot performance on these models, we aimed to create a single text prompt that had to include both the SPIDER-dev question as well as any schema needed for context.

Initially, we took a multi-stage prompting approach where models were given a chain of thought and prompted to:

- (1) read the schema and context given
- (2) sort out what components of the schema will be relevant to the question
- (3) build a SQL query based off the question
- (4) reevaluate the produced query for potential errors
- (5) return a single SQL query string based on SQLite's SQL dialect

However, this process would sometimes exceed the token window of the smaller models like Llama 3b or Qwen 4b, resulting in malformed returned strings, partial results, or other errors that made the results effectively unusable. To allow for one consistent

prompting strategy across all models regardless of size, we opted to use a very simple prompt, shown below.

Using this prompt, we were able to extract usable SQL the vast majority of the time from our models. Even though the prompt instructed the model to return only the SQL query, some models were not able to follow this instruction. As such we added a REGEX based extraction feature to our testbench, which was able to extract SQL when the model returned a query string in most cases, such as when the string is preceded by a newline character, or when it is formatted as SQL:<query>.

With these three design choices in mind, we were able to use Novita AI's cloud AI service to evaluate our selected models on the SPIDER-dev dataset, we were able to run our desired tests and gather some meaningful insights into how the models behave when faced with a variety of NL2SQL tasks.

4 RESULTS

In testing, each of the selected models was faced with all 1,034 of the SPIDER-dev dataset questions. We measured three primary metrics - completion rate (whether or not the model returned parsable SQL), test accuracy (predicted SQL successfully completing the query correctly), and exact match accuracy (predicted SQL perfectly matching the benchmark SQL string).

5 ANALYSIS

6 CONCLUSION

REFERENCES

- [1] DeepSeek AI. 2025. deepseek/deepseek-r1-distill-qwen-14b. <https://huggingface.co/deepseek/deepseek-r1-distill-qwen-14b>. As of Dec 2025.
- [2] Novita AI. 2025. Novita AI. <https://novita.ai/>. As of Dec 2025.
- [3] Alibaba Cloud. 2025. qwen/qwen2.5-7b-instruct. <https://huggingface.co/qwen/qwen2.5-7b-instruct>. As of Dec 2025.
- [4] Alibaba Cloud. 2025. qwen/qwen3-4b-fp8. <https://huggingface.co/qwen/qwen3-4b-fp8>. As of Dec 2025.
- [5] Alibaba Cloud. 2025. qwen/qwen3-8b-fp8. <https://huggingface.co/qwen/qwen3-8b-fp8>. As of Dec 2025.
- [6] A. Corradini, F. Martino, and P. Rossi. 2025. A Comprehensive Survey of Small Language Models in the Era of Large Language Models. *Big Data and Cognitive Computing* 9, 7 (2025), 189. <https://doi.org/10.3390/bdcc9070189>
- [7] Google. 2025. google/gemma-3-12b-it. <https://huggingface.co/gemma-ai/gemma-3-12b>. As of Dec 2025.
- [8] Google. 2025. google/gemma-3-27b-it. <https://huggingface.co/google/gemma-3-27b-it>. As of Dec 2025.
- [9] T. Hasan, Y. Li, and M. Chen. 2025. Assessing Small Language Models for Code Generation. *arXiv preprint arXiv:2507.03160* preprint (2025), 1 – 17. <https://arxiv.org/abs/2507.03160>
- [10] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. *arXiv:2411.07763* [cs.CL] <https://arxiv.org/abs/2411.07763>
- [11] Boyan Li, Yuyu Luo, Nan Tang, Guoliang Li, and Chengliang Chai. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? *Vldb* 17, 11 (2024), 3318 – 3331. <https://www.vldb.org/pvldb/vol17/p3318-luo.pdf>
- [12] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. *arXiv:2305.03111* [cs.CL] <https://arxiv.org/abs/2305.03111>
- [13] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109* 18, 9 (2024), 1 – 20. <https://doi.org/10.48550/arXiv.2408.05109>
- [14] llm stats.com. 2025. LLM Leaderboard. <https://llm-stats.com/>. As of Dec 2025.
- [15] Meta. 2025. meta-llama/llama-3.1-8b-instruct. <https://huggingface.co/meta-llama/llama-3.1-8b-instruct>. As of Dec 2025.
- [16] Meta. 2025. meta-llama/llama-3.2-3b-instruct. <https://huggingface.co/meta-llama/llama-3.2-3b-instruct>. As of Dec 2025.
- [17] OpenAI. 2025. openai/gpt-oss-20b. <https://huggingface.co/openai/gpt-oss-20b>. As of Dec 2025.
- [18] Wenqi Pei, Hailing Xu, Hengyuan Zhao, Shizheng Hou, Han Chen, Zining Zhang, Pingyi Luo, and Bingsheng He. 2025. Feather-SQL: A Lightweight NL2SQL Framework with Dual-Model Collaboration Paradigm for Small Language Models. *arXiv preprint*, 17811 (2025), 1 – 20. <https://arxiv.org/abs/2503.17811>
- [19] GitHub Repository. 2025. Source Code for Experiment Report. <https://github.com/CATBOYCOWBOY/comp5118Final.git>.
- [20] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanell Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium.
- [21] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanell Roman, Zilin Zhang, and Dragomir Radev. 2019. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *arXiv:1809.08887* [cs.CL] <https://arxiv.org/abs/1809.08887>
- [22] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suite. In *The 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

You are an expert SQL analyst. Generate a SQL query to answer the question based on the provided database schema.

<CONTEXT/SCHEMA>

Question: <QUESTION - FROM SPIDER-DEV DATASET>.

Requirements:

- Use proper SQL syntax for SQLite
- Return ONLY the final SQL query
- Do NOT include any explanations, reasoning, or additional text
- Do NOT wrap the query in backticks, code blocks, or markdown formatting
- Do NOT include prefixes like "sql:", "query:", or similar

SQL Query:

Figure 1: Zero-shot prompting template used for model tests